Programmieren 1

1. Tutorium · Grundlagen, Terminal, Hello World & Variablen

Tutorium01.java

```
public class Tutorium01 {
    public static void main(String[] args) {
        System.out.println("Schön, dass ihr da seid!");
    }
}
```

Willkommen! Wer steht hier vorne?

Steckbrief

Name: Arjuna Matev

Kontakt: matev@campus.tu-berlin.de

Fragen: Im Tutorium, per Mail oder im Forum

Willkommen! Wer steht hier vorne?

Steckbrief

Name: Arjuna Matev

Kontakt: matev@campus.tu-berlin.de

Fragen: Im Tutorium, per Mail oder im Forum

Fahrplan für heute

1. **Terminal:** Navigation & Dateien anlegen
2. **Hello World:** Kompilieren (javac) & Starten (java)
3. **IntelliJ IDEA:** Projekt-Setup & Programm-Aufbau
4. **Variablen & Literale:** Datentypen & Live-Quiz

Wie läuft unser Tutorium ab?

Was wir machen

- Live zusammen coden
- Kurze Praxis-Übungen
- Eure Fragen klären

Wie läuft unser Tutorium ab?

Was wir machen

- Live zusammen coden
- Kurze Praxis-Übungen
- Eure Fragen klären

Mitmachen

- Ich frage regelmäßig in die Runde
- Kein Stress: Passen ist jederzeit okay!
- Sofort reinrufen bei Fragen

Wie läuft unser Tutorium ab?

Was wir machen

- Live zusammen coden
- Kurze Praxis-Übungen
- Eure Fragen klären

Mitmachen

- Ich frage regelmäßig in die Runde
- Kein Stress: Passen ist jederzeit okay!
- Sofort reinrufen bei Fragen

Was wir nicht sind

- Keine 2. Vorlesung
- Kein Vorgriff auf neuen Stoff
- Kein Ersatz fürs Selbst-Tippen

Wie kommt man gut durch Prog 1?

Programmieren ist Handwerk

Ungewohnt am Anfang: Völlig normal ohne Vorkenntnisse!

Jede Woche dranbleiben: Kontinuität schlägt Bulimie-Lernen

Selbst tippen: Zuschauen allein reicht nicht

Wie kommt man gut durch Prog 1?

Programmieren ist Handwerk

Ungewohnt am Anfang: Völlig normal ohne Vorkenntnisse!

Jede Woche dranbleiben: Kontinuität schlägt Bulimie-Lernen

Selbst tippen: Zuschauen allein reicht nicht

Ehrlicher Talk zu ChatGPT 🤖

KI löst Prog 1 leicht: Aber ihr nehmt dabei nichts mit

In der Klausur: Sitzt ihr ohne KI vor dem Code

Faustregel: Wenn es sich anstrengend anfühlt, lernt euer Kopf gerade!

Teil 1: Warum das Terminal?

GUI vs. Shell

GUI: Fenster, Maus & Menüs

Shell (Terminal): Textbefehle direkt ans Betriebssystem

Linux / macOS: Bash / Zsh

Windows: PowerShell

Teil 1: Warum das Terminal?

GUI vs. Shell

GUI: Fenster, Maus & Menüs

Shell (Terminal): Textbefehle direkt ans Betriebssystem

Linux / macOS: Bash / Zsh

Windows: PowerShell

Terminal-Demo

```
# Aktueller Ordner:
$ pwd
/home/studi/Uni

# Ordner anlegen & wechseln:
$ mkdir prog1_tutorium
$ cd prog1_tutorium

# Datei erstellen:
$ touch HelloWorld.java
```

ZUM MITMACHEN · AUFGABE 2 & 3

Cheat-Sheet: Wichtige Terminal-Befehle

Aktion	Linux / macOS (Bash)	Windows (PowerShell)
Wo bin ich?	`pwd`	`Get-Location` (oder `pwd`)
Inhalt anzeigen	`ls`	`Get-ChildItem` (oder `ls`)
Ordner wechseln	`cd <Ordner>`	`cd <Ordner>`
Ordner anlegen	`mkdir prog1_tutorium`	`mkdir prog1_tutorium`
Datei anlegen	`touch HelloWorld.java`	`New-Item -ItemType File HelloWorld.java`
Editor öffnen	`nano HelloWorld.java`	`notepad HelloWorld.java`
Hilfe lesen	`man <Befehl>` (q = Ende)	`Get-Help <Befehl>`

Vom Code zum Programm: javac & java

1. Quellcode

HelloWorld.java

Von uns geschriebener Text

Terminal

```
# Schritt 0: Ist das Java Development Kit (JDK) da?
$ java -version
```

Vom Code zum Programm: javac & java

1. Quellcode

HelloWorld.java

Von uns geschriebener Text

2. Compiler (javac)

javac HelloWorld.java

Übersetzt in Bytecode (.class)

Terminal

```
# Schritt 1: Kompilieren (mit Dateiendung .java!)
$ javac HelloWorld.java
$ ls
HelloWorld.class   HelloWorld.java
```

Vom Code zum Programm: javac & java

1. Quellcode

HelloWorld.java

Von uns geschriebener Text

2. Compiler (javac)

javac HelloWorld.java

Übersetzt in Bytecode (.class)

3. JVM starten (java)

java HelloWorld

Führt Bytecode aus

Terminal

```
# Schritt 2: Ausführen (OHNE .java oder .class am Ende!)
$ java HelloWorld
Hello, World!
```

Anatomie von HelloWorld.java

HelloWorld.java

```java
public class HelloWorld {
  public static void main(String[] args) {
    // Gibt Text aus:
    System.out.println("Hello, World!");
  }
}
```

1. Die Klasse (class)

public class HelloWorld

- **Goldene Regel:** Dateiname MUSS exakt wie die Klasse heißen (HelloWorld.java)!
- **Block { }:** Alles zwischen { und } gehört zur Klasse.

Anatomie von HelloWorld.java

HelloWorld.java

```
public class HelloWorld {
  public static void main(String[] args) {
    // Gibt Text aus:
    System.out.println("Hello, World!");
  }
}
```

2. Die main-Methode

public static void main(String[] args)

- **Der Startpunkt:** Hier beginnt Java IMMER mit der Ausführung.
- **Fürs Erste:** Einfach exakt so in jedes Programm übernehmen!

Anatomie von HelloWorld.java

HelloWorld.java

```
public class HelloWorld {
  public static void main(String[] args) {
    // Gibt Text aus:
    System.out.println("Hello, World!");
  }
}
```

3. Kommentare

// Kommentar bis Zeilenende

/* … */ für mehrzeilige Kommentare

- **Für den Compiler unsichtbar:** Perfekt für Notizen oder zum Ausblenden von Code.

Anatomie von HelloWorld.java

HelloWorld.java

```
public class HelloWorld {
  public static void main(String[] args) {
    // Gibt Text aus:
    System.out.println("Hello, World!");
  }
}
```

4. Anweisung (Statement) & Ausgabe

System.out.println("Hello, World!");

- **Semikolon ;** beendet jede Anweisung in Java!
- **println vs. print:** println macht am Ende einen Zeilenumbruch, print nicht.

Teil 2: Was ist eine Variable?

Technisch: Speicher im RAM

Name für Speicheradresse: Wir müssen uns keine Bit-Adressen merken

Garbage Collector: Java räumt nicht mehr genutzten Speicher selbst auf

Streng typisiert: Wir müssen Java die Größe vorab nennen -> Datentyp!

Teil 2: Was ist eine Variable?

Technisch: Speicher im RAM

Name für Speicheradresse: Wir müssen uns keine Bit-Adressen merken

Garbage Collector: Java räumt nicht mehr genutzten Speicher selbst auf

Streng typisiert: Wir müssen Java die Größe vorab nennen -> Datentyp!

Bildlich: Der Pappkarton 📦

Etikett = Variablenname

Kartongröße = Datentyp (Schuhkarton vs. Kühlschrank)

Reinlegen = Wert zuweisen (=)

Reinschauen = Wert auslesen

Die 8 primitiven Datentypen (+ String)

Typ	Größe	Inhalt	Wertebereich	Beispiel
`byte`	8 Bit	Ganze Zahl	[-128 ; 127]	`42`
`short`	16 Bit	Ganze Zahl	[-32.768 ; 32.767]	`1000`
`int (Standard)`	**32 Bit**	**Ganze Zahl**	**ca. ±2 Milliarden**	**`123_456`**
`long`	64 Bit	Ganze Zahl	$[-2^{63} ; 2^{63} - 1]$	`9999999999L`

Die 8 primitiven Datentypen (+ String)

Typ	Größe	Inhalt	Wertebereich	Beispiel
`byte`	8 Bit	Ganze Zahl	[-128 ; 127]	`42`
`short`	16 Bit	Ganze Zahl	[-32.768 ; 32.767]	`1000`
`int (Standard)`	**32 Bit**	**Ganze Zahl**	**ca. ±2 Milliarden**	**`123_456`**
`long`	64 Bit	Ganze Zahl	$[-2^{63} ; 2^{63} - 1]$	`9999999999L`
`float`	32 Bit	Kommazahl	Einfache Genauigkeit	`3.14f`
`double (Standard)`	**64 Bit**	**Kommazahl**	**Doppelte Genauigkeit**	**`3.14159`**

Die 8 primitiven Datentypen (+ String)

Typ	Größe	Inhalt	Wertebereich	Beispiel
`byte`	8 Bit	Ganze Zahl	[-128 ; 127]	`42`
`short`	16 Bit	Ganze Zahl	[-32.768 ; 32.767]	`1000`
`int (Standard)`	**32 Bit**	**Ganze Zahl**	**ca. ±2 Milliarden**	**`123_456`**
`long`	64 Bit	Ganze Zahl	$[-2^{63} ; 2^{63} - 1]$	`9999999999L`
`float`	32 Bit	Kommazahl	Einfache Genauigkeit	`3.14f`
`double (Standard)`	**64 Bit**	**Kommazahl**	**Doppelte Genauigkeit**	**`3.14159`**
`boolean`	1 Bit*	Wahrheitswert	true / false	`true`
`char`	16 Bit	Einzelzeichen	Unicode-Zeichen	`'A', '\n'`

Die 8 primitiven Datentypen (+ String)

Typ	Größe	Inhalt	Wertebereich	Beispiel
`byte`	8 Bit	Ganze Zahl	[-128 ; 127]	`42`
`short`	16 Bit	Ganze Zahl	[-32.768 ; 32.767]	`1000`
`int (Standard)`	**32 Bit**	**Ganze Zahl**	**ca. ±2 Milliarden**	**`123_456`**
`long`	64 Bit	Ganze Zahl	$[-2^{63} ; 2^{63} - 1]$	`9999999999L`
`float`	32 Bit	Kommazahl	Einfache Genauigkeit	`3.14f`
`double (Standard)`	**64 Bit**	**Kommazahl**	**Doppelte Genauigkeit**	**`3.14159`**
`boolean`	1 Bit*	Wahrheitswert	true / false	`true`
`char`	16 Bit	Einzelzeichen	Unicode-Zeichen	`'A', '\n'`
`String*`	**variabel**	**Text (Referenztyp!)**	**Zeichenkette**	**`"Hallo"`**

Variablen anlegen & benennen

MyVariables.java

```
// 1. Deklaration (Karton aufstellen):
int myInt;
```

Variablen anlegen & benennen

MyVariables.java

```java
// 1. Deklaration (Karton aufstellen):
int myInt;

// 2. Zuweisung (Wert reinlegen mit =):
myInt = 5;

// Oder direkt in einer Zeile:
int studentCount = 120;
```

Variablen anlegen & benennen

MyVariables.java

```java
// 1. Deklaration (Karton aufstellen):
int myInt;

// 2. Zuweisung (Wert reinlegen mit =):
myInt = 5;

// Oder direkt in einer Zeile:
int studentCount = 120;
```

Namensregeln (Identifier)

Erlaubt: Buchstaben, Zahlen, _ und $

Wichtig: Darf NICHT mit einer Ziffer starten!

Verboten: Java-Keywords (int, class, true...)

Style: lowerCamelCase & sprechende Namen

Literale (1/2): Feste Zahlenwerte im Code

Ganzzahl-Literale

Standardtyp: Immer int (32 Bit)

Für long: Suffix L anhängen (z.B. 42L)

Extras: Unterstriche (1_000), Binär (0b), Hex (0x)

Ganzzahlen

```
int million = 1_000_000;
long big    = 9999999999L; // L Pflicht!
int bin     = 0b1011;      // = 11
```

Literale (1/2): Feste Zahlenwerte im Code

Ganzzahl-Literale

Standardtyp: Immer int (32 Bit)

Für long: Suffix L anhängen (z.B. 42L)

Extras: Unterstriche (1_000), Binär (0b), Hex (0x)

Ganzzahlen

```
int million = 1_000_000;
long big    = 9999999999L; // L Pflicht!
int bin     = 0b1011;      // = 11
```

Kommazahl-Literale

Standardtyp: Immer double (64 Bit, Punkt statt Komma!)

Für float: Suffix f anhängen (z.B. 3.14f)

10er-Potenzen: Wissenschaftlich mit e (1.3e4)

Gleitkommazahlen

```
double d = 3.1415; // Standard: double
float  f = 3.14f;  // Ohne f -> Fehler!
double s = 1.3e4;  // = 1.3 * 10^4
```

Literale (2/2): Zeichen, Text & Wahrheitswerte

char vs. String

char: Einfache Anführungszeichen 'a'

Escape-Sequenzen: '\n' (Umbruch), '\t' (Tab), '\\'

String: Doppelte Anführungszeichen "Text"

char & String

```
char c   = 'A';
char tab = '\t';
String s = "Zeile 1\nZeile 2";
```

Literale (2/2): Zeichen, Text & Wahrheitswerte

char vs. String

char: Einfache Anführungszeichen 'a'

Escape-Sequenzen: '\n' (Umbruch), '\t' (Tab), '\\'

String: Doppelte Anführungszeichen "Text"

char & String

```
char c   = 'A';
char tab = '\t';
String s = "Zeile 1\nZeile 2";
```

boolean & null

boolean: Nur true oder false (nicht 0/1!)

null-Literal: Leere Referenz (für String)

Wichtig: Lokale Variablen immer initialisieren!

boolean & null

```
boolean isReady = true;
boolean hasBug  = false;
String empty    = null;
```

Sichtbarkeit von Variablen (Scope)

ScopeDemo.java

```
public static void main(String[] args) { // Block A
  int a = 10; // sichtbar in A, B und C

  { // Unterblock B
    int b = 20; // nur in B sichtbar!
    // int a = 99; -> FEHLER (existiert schon)
  }
  { // Geschwisterblock C
    int b = 30; // OK! (B ist schon beendet)
  }
}
```

Sichtbarkeit von Variablen (Scope)

ScopeDemo.java

```
public static void main(String[] args) { // Block A
  int a = 10; // sichtbar in A, B und C

  { // Unterblock B
    int b = 20; // nur in B sichtbar!
    // int a = 99; -> FEHLER (existiert schon)
  }
  { // Geschwisterblock C
    int b = 30; // OK! (B ist schon beendet)
  }
}
```

Die 3 Faustregeln

1. Sichtbarkeit: Gilt im eigenen { }-Block und allen Unterblöcken.

2. Kein Überschreiben: Unterblock darf Namen aus Elternblock nicht neu vergeben.

3. Geschwisterblöcke: Kennen sich nicht -> Name darf neu genutzt werden.

Live-Quiz: Spielt mal Java-Compiler!

CompilerQuiz.java

```
// Welche Zeilen kompilieren? Wo meckert javac?

1   int population = 84_000_000;
2   float temp = 21.5;
3   char grade = "A";
4   int 1stPlace = 100;
5   long worldPop = 8_000_000_000;
6   byte small = 200;
```

Was meint ihr?

Schaut euch Zeile 1 bis 6 genau an:

- Welche Zeilen sind gültig?
- Wie müssen wir die fehlerhaften Zeilen reparieren?

Live-Quiz: Spielt mal Java-Compiler!

CompilerQuiz.java

```
// Welche Zeilen kompilieren? Wo meckert javac?

1   int population = 84_000_000;
2   float temp = 21.5;
3   char grade = "A";
4   int 1stPlace = 100;
5   long worldPop = 8_000_000_000;
6   byte small = 200;
```

Auflösung

Zeile 1: [OK] Unterstriche sind erlaubt

Zeile 2: [X] 21.5 ist double -> 21.5f

Zeile 3: [X] "A" ist String -> 'A'

Zeile 4: [X] Startet mit einer Ziffer

Zeile 5: [X] Zu groß für int -> L anhängen

Zeile 6: [X] byte geht nur bis 127

Geschafft für heute! 🎉

Noch offene Fragen oder Setup-Probleme?

- **Java / IntelliJ streikt noch?** Bleibt gerne kurz da, wir fixen das direkt!
- **Bis nächste Woche:** TUT1 Grundlagen fertig machen & Skript Kap. 1–2 anschauen.
- **Schönen Start ins Semester & bis nächste Woche!**